Unit B3: Object-Oriented Programming (Core)

Sub-topics: B3.1 Fundamentals of OOP for a single class

1. Syllabus Scope & Learning Outcomes

The "Do" column defines the practical skills required for the exam (Paper 1 & Paper 2).

Syllabus Point	Command Term	Student Expectation (The "Do")
B3.1.1	**Evaluate**	Discuss why OOP increases modularity, reusability, and encapsulation compared to procedural programming.
B3.1.2	**Construct**	Draw UML Class diagrams representing a class, its attributes, and methods.
B3.1.3	**Distinguish**	Differentiate between *Instance* variables (specific to one object) and *Static/Class* variables (shared by all).
B3.1.4	**Construct**	Write Python code to define a class, a constructor (__init__), and instantiate objects.
B3.1.5	**Explain / Apply**	Implement encapsulation using Python conventions (_ or __ prefix) and Accessor/Mutator methods.

2. Key Terminology & Definitions

- **Class:** A blueprint or template for creating objects (defines attributes and behaviors).

- **Object:** A specific instance of a class (e.g., my_car is an object of class Car).
- **Instantiation:** The process of creating an object from a class.
- **Constructor:** A special method (__init__ in Python) automatically called when an object is created to set initial values.
- **Encapsulation:** Hiding internal data and only allowing access through defined methods (getters/setters).
- **Instance Variable:** Data belonging to a specific object (e.g., self.name).
- **Static (Class) Variable:** Data shared by *all* objects of a class (e.g., next_id counter).
- **Accessor (Getter):** A method that returns the value of a private attribute.
- **Mutator (Setter):** A method that updates the value of a private attribute (often with validation).

3. Core Content & Exam Actions

B3.1.1: Fundamentals of OOP

Theory:

- **Modularity:** Objects are self-contained. Code changes in one class don't necessarily break others.
- **Abstraction:** The user uses the object methods without needing to know the internal code logic.
- **Blueprints:** Analogy that a Class is the architect's drawing; the Object is the actual house built from it.

Student Exam Action:

- **Evaluation:** If asked "Why use OOP for a library system?", explain how Book and User objects can be reused and managed independently.

B3.1.2: Designing Classes (UML)

Theory:

- **UML Structure:** A box split into three rows:
  1. **Class Name** (e.g., Student)
  2. **Attributes** (Variables) (e.g., - name: String)
  3. **Methods** (Functions) (e.g., + getName(): String)
- **Visibility Symbols:** + = Public, - = Private.

Student Exam Action:

- **Diagramming:** Given a scenario ("A car has a speed and color"), draw the standard 3-box UML diagram.

B3.1.3: Static vs Non-Static

Theory:

- **Instance (Non-Static):** self.variable. Unique to that specific object. Changing one

object's name doesn't change another's.
- **Class (Static):** Defined outside __init__. Shared by all. If you change it, it changes for everyone. Used for counters (e.g., total_students) or constants.

Student Exam Action:

- **Tracing:** Given code where a static variable increments every time an object is created, calculate the final value of that variable.

B3.1.4: Coding & Instantiation

Theory:

- **Definition:** class Dog:
- **Constructor:** def __init__(self, name):
- **Self:** Represents the specific object instance being accessed.
- **Instantiation:** my_dog = Dog("Rex")

Student Exam Action:

- **Coding:** Write a class Circle with a constructor that takes radius and a method area() that returns πr^2.

B3.1.5: Encapsulation

Theory:

- **Hiding Data:** In Python, prefix variables with _ (convention) or __ (name mangling) to indicate they are private.
- **Public Interface:** Provide get_balance() and deposit() methods rather than letting external code modify _balance directly.

Student Exam Action:

- **Refactoring:** Take "unsafe" code (e.g., p.age = -5) and rewrite it using a Setter method that validates the input (if age > 0).

4. Common Pitfalls & Examiner Tips

- **Pitfall (self):** Students often forget to include self as the first parameter in methods (def my_method(self):). Without it, the code fails.
- **Pitfall (UML):** Confusing the order of UML rows. Remember: **Name** -> **Attributes** -> **Methods**.
- **Tip (Constructors):** In Python, the constructor is *always* named __init__. Do not name it Class() or Constructor().
- **Tip (Static):** Use Class variables for generating unique IDs (e.g., ID = next_ID, then next_ID += 1).